

MCDI500

PROGRAMACIÓN PARA LA CIENCIA DE DATOS

ANÁLISIS DE CONDICIONES OPERACIONALES Y FALLAS CON

AI4I 2020

Informe técnico Proyecto final Fase 4

Nombre del estudiante: Alexander Sepúlveda Ormazábal

Grupo: 9

Docente responsable: Omar Salinas Silva

Fecha: 4 de octubre de 2026

Repositorio: github.com/alexander-seor/proyecto-grupo9-mcdi500

Índice

1	Introducción y contextualización.....	3
2	Definición de la problemática y objetivos.....	3
3	Aplicación de herramientas científicas	4
3.1	Reproducibilidad técnica	4
3.2	Control de versiones	4
3.3	Documentación del proceso	4
3.4	Repositorio GitHub actualizado	4
4	Diseño de soluciones algorítmicas eficientes	4
4.1	Codificación funcional	4
4.2	Preprocesamiento y transformación del dataset	4
4.3	Validación técnica y verificación del código	4
4.4	Eficiencia y optimización	5
4.5	Diseño estructurado del código	5
4.6	Notebook final ejecutado y documentado	5
5	Implementación de código modular y robusto	5
5.1	Programación orientada a objetos	5
5.2	Documentación de arquitectura	5
6	Construcción de visualizaciones efectivas	6
6.1	Preprocesamiento para visualización	6
6.2	Visualizaciones analíticas	6
7	Metodología del desarrollo técnico.....	9
7.1	Validación y reflexión técnica.....	9
7.2	Resultados	9
7.3	Discusión	9
7.4	Conclusiones	9
7.5	Trazabilidad de mejoras.....	10
8	Presentación audiovisual.....	10
9	Bibliografía	10

1 Introducción y contextualización

Las fallas inesperadas interrumpen la operación, reducen la productividad y aumentan las necesidades de mantenimiento. El proyecto estudia AI4I 2020, un conjunto sintético con 10.000 registros de temperatura, velocidad de rotación, torque, desgaste y tipo de producto, además de una variable binaria de falla (Matzka, 2020). El análisis busca reconocer condiciones que puedan orientar la priorización de inspecciones o monitoreo. Los resultados describen este conjunto y no constituyen umbrales industriales ni el desempeño de un modelo.

El cierre integra las cuatro fases. F1 delimitó el problema y el entorno; F2 preparó los datos; F3 reorganizó el pipeline con funciones, clases y módulos; F4 recuperó una versión interpretable, construyó las visualizaciones y comprobó los hallazgos en entrenamiento y prueba. Esta continuidad permite seguir cada cifra hasta su archivo de origen. En mantenimiento predictivo es necesario distinguir una asociación descriptiva del rendimiento de un modelo, especialmente cuando la clase de interés es poco frecuente (Phan et al., 2022).

2 Definición de la problemática y objetivos

La pregunta central es: ¿qué condiciones operacionales están asociadas con la ocurrencia de fallas? El objetivo general es analizar esa relación, identificar perfiles descriptivos de mayor frecuencia y dejar una base reproducible para una evaluación predictiva posterior. La entrega responde tres preguntas concretas: si existen diferencias por Type, qué rangos operacionales concentran más fallas y si el perfil obtenido con entrenamiento mantiene una diferencia semejante en prueba.

Los objetivos específicos abordados fueron:

- Caracterizar las variables operacionales, el tipo de producto y la variable de falla.
- Conservar un flujo reproducible desde el archivo original hasta los productos de F4.
- Explorar las tasas de falla por tipo, quintiles e interacción de condiciones.
- Comprobar en prueba el perfil definido con los datos de entrenamiento.

Alcance. El análisis considera las seis variables predictoras definidas en F1 y la variable Machine failure. Los identificadores y los cinco modos de falla permanecen excluidos para evitar fuga de información.

Restricciones. AI4I 2020 es sintético, contiene solo 339 fallas y no representa una planta industrial específica. El profesor autorizó continuar con este conjunto debido al avance existente. El cierre se limita a un análisis descriptivo y no entrena un clasificador.

Supuestos. Las unidades y definiciones publicadas por UCI se consideran válidas. Los registros se interpretan como observaciones independientes y los parámetros guardados por F3 corresponden al entrenamiento ejecutado.

3 Aplicación de herramientas científicas

NumPy controla la semilla y operaciones numéricas; pandas carga, agrupa y valida tablas; scikit-learn sustenta la división, One Hot y RobustScaler; Matplotlib y seaborn producen las figuras. Jupyter integra código, resultados y explicación en una secuencia ejecutable. Git registra la evolución del repositorio.

Herramienta	Versión	Uso principal
Python	3.13.0	Ejecución del proyecto
NumPy	2.5.3	Semilla y operaciones numéricas
pandas	3.0.5	Tablas, qcut y validación
Matplotlib	3.11.2	Composición de figuras
seaborn	0.13.2	Mapa de calor
scikit-learn	1.9.1	División y transformaciones

Tabla 1. Entorno científico registrado en la ejecución de F4

3.1 Reproducibilidad técnica

El notebook confirma que el intérprete pertenece a .venv, declara la semilla 42 y muestra las versiones. Las rutas se construyen desde la raíz del repositorio, por lo que no dependen de una carpeta personal. requirements.txt registra las dependencias y los metadatos conservan la fecha, las entradas y sus huellas SHA-256. La ejecución validada contiene 20 celdas de código numeradas del 1 al 20, sin errores; así, otra ejecución puede comprobar que utilizó los mismos archivos.

3.2 Control de versiones

El historial separa avances verificables: ed3da7b documenta F1, be64efc el preprocesamiento de F2, 858ba05 la modularización y eficiencia de F3, 5805bf0 su documentación, 2c0f565 los productos de F4 y 4de28f8 la trazabilidad final. El desarrollo es individual y autorizado; los commits registran la contribución real sin atribuir tareas a integrantes inexistentes.

3.3 Documentación del proceso

Cada fase conserva notebooks, diccionarios, parámetros, metadatos y productos procesados. El notebook de F4 añade tablas de respaldo para cada cifra, un mapa entre objetivos y figuras y una tabla de mejoras. README.md explica el orden de ejecución y F4/README.md describe las entradas, salidas y limitaciones.

3.4 Repositorio GitHub actualizado

La rama main organiza F1 a F4 y conserva el archivo original separado de los datos procesados. F4 contiene el notebook integrador, la copia interpretable, ocho tablas de apoyo, metadatos y cuatro imágenes. El repositorio se encuentra en github.com/alexander-seor/proyecto-grupo9-mcdi500.

4 Diseño de soluciones algorítmicas eficientes

4.1 Codificación funcional

Las funciones de carga, separación y medición reciben parámetros explícitos y devuelven resultados verificables. La alternativa funcional de F3 sirve como referencia para comprobar que la organización orientada a objetos no modifica el contenido.

4.2 Preprocesamiento y transformación del dataset

F2 comprobó 10.000 filas por 14 columnas, cero valores ausentes, cero duplicados exactos y tipos coherentes. No se inventaron errores ni se eliminaron filas porque el conjunto sintético no presentaba problemas reales de limpieza. UDI y Product ID se excluyeron por ser identificadores; TWF, HDF, PWF, OSF y RNF se retiraron para evitar fuga de información. Se conservaron Type y cinco mediciones, con Machine failure separada como objetivo.

La división estratificada produjo 8.000 registros de entrenamiento y 2.000 de prueba. One Hot Encoding convirtió Type en indicadores binarios y RobustScaler utilizó mediana e IQR para reducir la influencia de valores extremos. Ambos transformadores se ajustaron únicamente con entrenamiento y luego se aplicaron sin cambios a prueba. El resultado contiene nueve columnas y mantiene la variable objetivo al final.

4.3 Validación técnica y verificación del código

F3 comprueba un caso normal de cinco filas, un caso límite de una fila y dos excepciones: uso del pipeline antes del ajuste y categoría desconocida. Cada prueba tiene una salida verificable. assert_frame_equal confirmó que las matrices de F3 coinciden

con F2; F4 comprobó las 10.000 observaciones contra F1 después de recuperar unidades y categorías. Estas pruebas cubren funcionamiento, límites, errores previstos y equivalencia entre fases.

4.4 Eficiencia y optimización

Filas	Funcional (ms)	POO (ms)	Razón POO/funcional
1.000	3,16	9,42	2,98
5.000	4,46	18,31	4,10
10.000	4,65	12,75	2,74
20.000	6,89	14,60	2,12

Tabla 2. Crecimiento temporal medido con tamaños controlados

timeit repitió cada medición y se comparó el menor tiempo para reducir ruido; tracemalloc registró la memoria máxima. En el flujo completo, la alternativa funcional promedió 0,0133 s y 0,663 MiB, mientras la POO alcanzó 0,0373 s y 1,239 MiB. La funcional fue más rápida en todos los tamaños y ambas crecieron de forma compatible con $O(n)$. Se conservó la arquitectura modular por su mantenibilidad; la optimización futura consiste en reducir copias y agrupar transformaciones numéricas.

4.5 Diseño estructurado del código

La cohesión es alta porque carga.py, transformadores.py, pipeline.py, fabrica.py y medicion.py tienen responsabilidades específicas. El acoplamiento es bajo porque Pipeline recorre una lista de objetos mediante ajustar y transformar, sin depender de cada implementación. El flujo usa un ciclo porque los pasos son lineales; una llamada recursiva agregaría complejidad sin beneficio. La recursividad se reservó para aplanar(), que recorre metadatos anidados en F2.

4.6 Notebook final ejecutado y documentado

F4 integra los cuatro productos de transición, la evidencia de pruebas y eficiencia, la reconstrucción interpretable, el análisis, las figuras y la verificación final. Cada bloque de código tiene una explicación previa y todas las salidas necesarias quedan visibles.

5 Implementación de código modular y robusto

5.1 Programación orientada a objetos

Transformador define el contrato común; CodificadorOneHot y EscaladorRobusto heredan ese comportamiento y aplican tareas distintas. El pipeline usa polimorfismo al llamar los mismos métodos en cada objeto. Los atributos _parametros y _ajustado encapsulan el estado y las guardas impiden transformar antes de ajustar.

5.2 Documentación de arquitectura

FabricaTransformadores concentra la creación de objetos según el tipo de variable. El problema era evitar que Pipeline conociera cómo construir cada clase; Factory resuelve esa creación y permite agregar un transformador cambiando la configuración, sin reescribir el orquestador. La consecuencia es una arquitectura más extensible, aunque incorpora más objetos que la versión funcional.

Componente	Responsabilidad	Archivo
Transformador	Contrato común y estado	transformadores.py
Codificador / Escalador	Transformaciones concretas	transformadores.py
Pipeline	Ejecuta pasos en orden	pipeline.py
FabricaTransformadores	Crea objetos configurados	fabrica.py
medir_recursos	Mide tiempo y memoria	medicion.py

Tabla 3. Componentes principales de la arquitectura de F3

```
pipeline = FabricaTransformadores.crear(configuracion)
pipeline.ajustar(datos_entrenamiento)
entrenamiento = pipeline.transformar(datos_entrenamiento)
prueba = pipeline.transformar(datos_prueba) # mismos parámetros
verificar_equivalencia(entrenamiento, referencia_f2)
```

Fragmento 1. Flujo de ajuste con entrenamiento y aplicación sin cambios a prueba

6 Construcción de visualizaciones efectivas

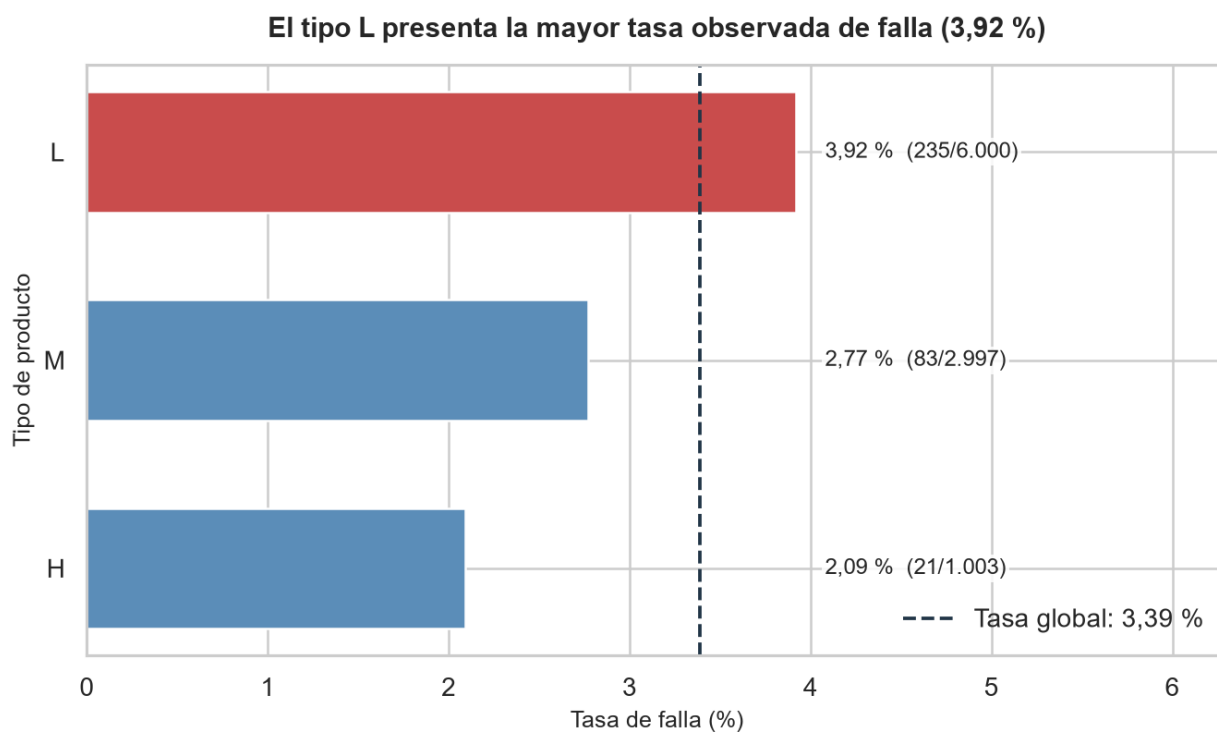
6.1 Preprocesamiento para visualización

Los CSV de F3 son apropiados para algoritmos, pero sus valores numéricos están escalados y Type aparece en tres columnas. F4 recupera cada medición con $\text{original} = \text{escalado} \times \text{IQR} + \text{mediana}$ y reconstruye H, L o M desde la única columna One Hot activa. La copia mantiene una columna de partición y coincide con las siete variables analíticas de F1.

Partición	Grupo	Fallas / registros	Tasa
Entrenamiento	Perfil combinado	79 / 273	28,94 %
Entrenamiento	Resto	192 / 7.727	2,48 %
Prueba	Perfil combinado	22 / 80	27,50 %
Prueba	Resto	46 / 1.920	2,40 %

Tabla 4. Validación descriptiva del perfil combinado por partición

6.2 Visualizaciones analíticas

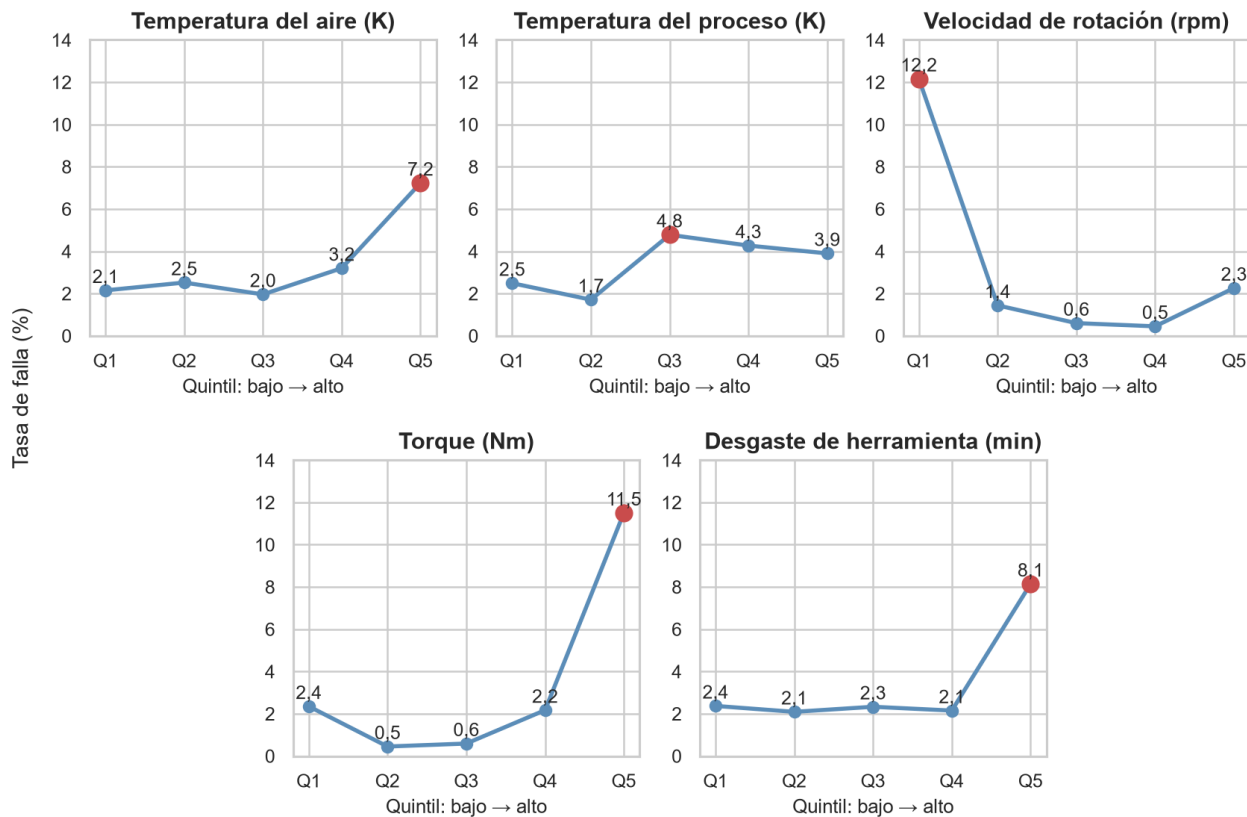


Fuente: Elaboración propia a partir de AI4I 2020 (Matzka, 2020).

Figura 1. El tipo L presenta la mayor tasa observada de falla

La Figura 1 entrega el contexto categórico. L registra 235 fallas entre 6.000 observaciones y una tasa de 3,92 %, superior a la tasa global de 3,39 %. M alcanza 2,77 % y H 2,09 %. Los tamaños de los grupos son distintos y la comparación no demuestra causalidad.

Las tasas más altas aparecen con velocidad baja (12,2 %) y torque alto (11,5 %)

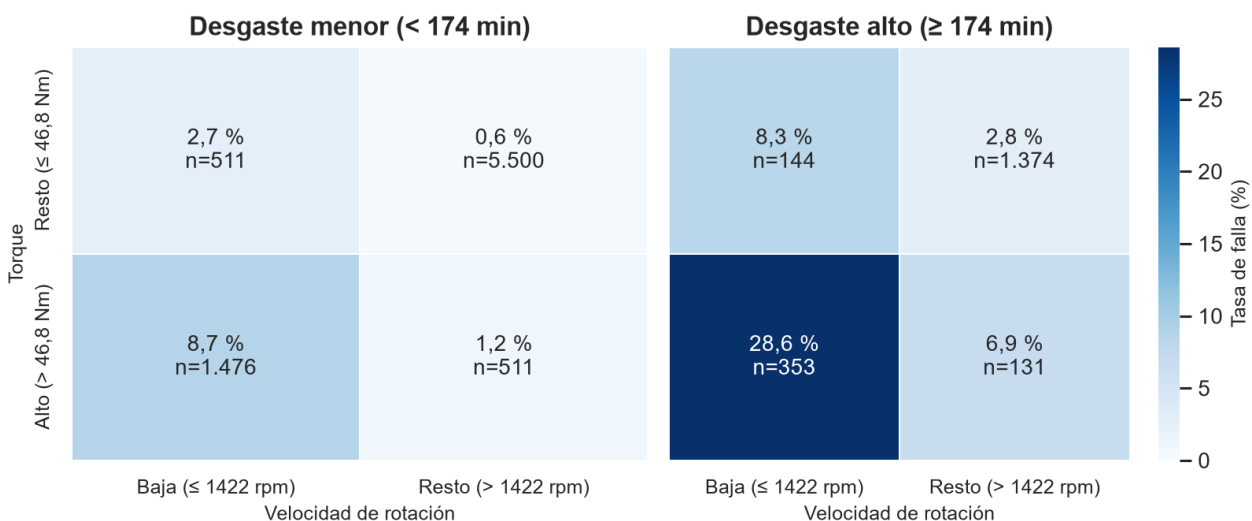


Fuente: elaboración propia a partir de AI4I 2020 (Matzka, 2020). Q1 representa valores bajos y Q5 valores altos; los límites están en la tabla de respaldo.

Figura 2. Las tasas más altas por quintil aparecen con velocidad baja y torque alto

La Figura 2 contrasta las cinco variables en grupos de tamaño parecido. La velocidad alcanza 12,16 % en Q1 y el torque 11,50 % en Q5. El desgaste y la temperatura del aire también aumentan en Q5, mientras la temperatura del proceso no sigue un recorrido creciente. Los intervalos se calcularon con `pandas.qcut` y describen este conjunto; no son límites industriales (`pandas development team`, 2026).

Con torque alto y velocidad baja, la tasa alcanza 28,6 % cuando el desgaste es alto



Fuente: elaboración propia a partir de AI4I 2020 (Matzka, 2020). Los cortes se obtienen del entrenamiento y no representan límites industriales.

Figura 3. La interacción de velocidad baja, torque alto y desgaste alto concentra la mayor tasa

La Figura 3 resuelve la pregunta al reunir tres condiciones. Con cortes calculados desde entrenamiento, el segmento completo contiene 101 fallas entre 353 registros y alcanza 28,61 %. El mapa se construyó con una escala común y cifras dentro de cada celda para que la lectura no dependa solo del color (seaborn development team, 2026).

La prueba separada mantiene la diferencia: 28,94 % en entrenamiento y 27,50 % en prueba para el perfil, frente a 2,48 % y 2,40 % en el resto. Esta cercanía fortalece la observación descriptiva. No corresponde a exactitud, sensibilidad ni otra métrica predictiva.

7 Metodología del desarrollo técnico

La metodología articula productos verificables de F1 a F4. F1 fijó la pregunta, las variables y el entorno; F2 revisó integridad, dividió 80/20 y ajustó las transformaciones solo con entrenamiento; F3 reprodujo el flujo con funciones y objetos, comparó equivalencia y midió eficiencia; F4 invirtió el escalamiento, reconstruyó Type y contrastó las mismas filas contra el archivo original. Git separó estos avances en commits y el changelog relaciona cada mejora con su evidencia.

Las decisiones técnicas siguieron el objetivo analítico. Se excluyeron identificadores y modos de falla para evitar señales impropias; RobustScaler se eligió por su resistencia a valores extremos; el ciclo del pipeline mantuvo claro un flujo lineal y la recursividad se utilizó solo para metadatos anidados. Los cortes del perfil combinado se calcularon con entrenamiento y se aplicaron sin cambios a prueba. Las mediciones mostraron que la versión funcional consume menos recursos, pero se conservó la POO por su mantenibilidad. Las figuras se ordenaron como contexto, contraste y resolución (Universidad Andrés Bello, 2026a, 2026b).

Fase	Decisión técnica	Evidencia	Impacto
F1	Definir problema, variables y entorno	F1_Definicion.ipynb	Base analítica reproducible
F2	Dividir antes de ajustar transformaciones	CSV procesados	Prevención de fuga
F3	Modularizar, probar y medir	Pruebas y benchmark	Equivalencia y mantenibilidad
F4	Recuperar unidades y contrastar perfil	Tablas y figuras	Resultado interpretable

Tabla 5. Relación entre fases, decisiones y resultados verificables

7.1 Validación y reflexión técnica

F2 verificó calidad y fuga de información; F3 añadió casos normal, límite y excepciones, además de equivalencia exacta con F2; F4 comprobó reconstrucción, archivos y correspondencia entre objetivos y figuras. El paso desde un notebook funcional a módulos POO aumentó claridad y escalabilidad, aunque elevó tiempo y memoria. El perfil surgió después de explorar los datos, por lo que su contraste en prueba confirma una diferencia descriptiva y no valida un modelo. El trabajo individual quedó respaldado por el historial real de Git.

7.2 Resultados

- Caracterización: la falla aparece en 339 de 10.000 registros, equivalentes al 3,39 %.
- Comparación por tipo: L alcanza 3,92 %, frente a 2,77 % en M y 2,09 % en H.
- Rangos operacionales: velocidad baja y torque alto presentan los extremos individuales más marcados.
- Estabilidad descriptiva: el perfil combinado registra 28,94 % en entrenamiento y 27,50 % en prueba.

7.3 Discusión

La expectativa inicial de encontrar condiciones con distinta frecuencia se sostiene para velocidad baja, torque alto y desgaste elevado, pero no para la temperatura de proceso, que no muestra una tendencia creciente uniforme. La asociación no demuestra causalidad. Los principales riesgos son el carácter sintético de AI4I 2020, el desbalance de la clase, los cortes dependientes del conjunto y la inspección de prueba durante esta confirmación descriptiva. Un estudio predictivo deberá definir el modelo antes de revisar validación y reportar sensibilidad, precisión, F1 y PR-AUC.

7.4 Conclusiones

El proyecto cumplió el objetivo descriptivo al identificar asociaciones reproducibles entre la ocurrencia de fallas y las condiciones operacionales. La combinación de velocidad baja, torque alto y desgaste elevado alcanza 28,94 % en entrenamiento y 27,50 % en prueba, frente a 2,48 % y 2,40 % en los demás registros. La cercanía entre ambas particiones sostiene la utilidad descriptiva del perfil dentro de AI4I 2020, aunque no demuestra causalidad, capacidad predictiva ni validez como umbral industrial.

El desarrollo también mostró que una solución de ciencia de datos no depende solo del resultado final. La separación previa de entrenamiento y prueba evitó fuga de información; las pruebas confirmaron la equivalencia entre F2 y F3; y la reconstrucción de F4 permitió comunicar valores comprensibles. La versión funcional fue más eficiente, mientras la organización POO facilitó mantenimiento y extensión. Como continuación, se requiere contrastar el perfil con datos reales, definir el clasificador antes de revisar la validación y evaluar sensibilidad, precisión, F1 y PR-AUC. Estas mejoras responden al desbalance de la clase y a las limitaciones del conjunto sintético.

7.5 Trazabilidad de mejoras

Observación	Mejora e impacto técnico	Commit
AI4I es sintético y no presenta limpieza real	No se inventaron errores; mejora la transparencia metodológica	be64efc
Evitar fuga de información	Se dividió antes de ajustar y se excluyeron modos de falla	be64efc
Modularizar y medir el código	Clases, Factory, pruebas y benchmark mejoraron mantenibilidad	858ba05
Datos escalados poco interpretables	Se recuperaron unidades y Type para comunicar resultados	2c0f565
Completar trazabilidad final	Changelog y metadatos vinculan archivos, decisiones y commits	4de28f8

Tabla 6. Vinculación entre observaciones, mejoras y evidencia verificable

El archivo changelog.md relaciona las mejoras con commits reales del repositorio. Las huellas de los cuatro archivos recibidos desde F3 quedaron registradas en metadatos_fase4.json.

8 Presentación audiovisual

La presentación se organiza para una duración de 6 a 7 minutos y participación individual. El apoyo visual muestra el problema, la continuidad entre fases, las tres figuras principales, la validación en prueba y los límites. El guion utiliza frases breves para explicar las cifras sin leer el informe ni atribuir capacidad predictiva.

9 Bibliografía

Matzka, S. (2020). *AI4I 2020 Predictive Maintenance Dataset* [Conjunto de datos]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5HS5C>

pandas development team. (2026). *pandas.qcut*. pandas documentation. <https://pandas.pydata.org/docs/reference/api/pandas.qcut.html>

Phan, T. L. J., Gehrhardt, I., Heik, D., Bahrpeyma, F., & Reichelt, D. (2022). A systematic mapping study on machine learning techniques applied for condition monitoring and predictive maintenance in the manufacturing sector. *Logistics*, 6(2), 35. <https://doi.org/10.3390/logistics6020035>

seaborn development team. (2026). *seaborn.heatmap*. seaborn documentation. <https://seaborn.pydata.org/generated/seaborn.heatmap.html>

Universidad Andrés Bello. (2026a). *Guía de apoyo Sumativa 3 Fase 4* [Documento de apoyo docente]. MCDI500 Programación para la Ciencia de Datos.

Universidad Andrés Bello. (2026b). *MCDI500 Apunte Fase 4* [Material de clase]. MCDI500 Programación para la Ciencia de Datos.